

Universidade Federal do Amazonas

Disciplina: Processamento Digital de Imagens

Grupo de Pesquisa em Reconhecimento de Padrões e Otimização –
GPRP&O

Projeto #6

`twodSFilter` & `medianSFilter`

Matheus Serrão Uchôa
Manaus, 26 de maio de 2026

1 Objetivo

Este projeto implementa duas funções de filtragem espacial 2-D em MATLAB:

1. `twodSFilter(f, w)` — filtragem espacial linear (correlação 2-D) com padding replicado e normalização da entrada para $[0, 1]$. Testada com filtro de média (*box filter*) de tamanhos 3×3 , 11×11 e 21×21 sobre a imagem `Fig3.37(a).jpg`.
2. `medianSFilter(f, w)` — filtragem espacial pela mediana (operação não-linear) com padding replicado. Testada com vizinhança 3×3 sobre a mesma imagem.

2 Fundamentação Teórica

2.1 Filtragem Espacial Linear — Correlação 2-D

A filtragem espacial linear de uma imagem $f(x, y)$ por um *kernel* $w(s, t)$ de tamanho $(2a + 1) \times (2b + 1)$ é definida pela operação de correlação:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t) \quad (1)$$

A convolução difere da correlação pelo sinal dos índices $(x - s)$, mas para kernels simétricos (como o filtro de média) as operações são equivalentes.

2.2 Filtro de Média (*Box Filter*) — Passa-Baixa

O filtro de média utiliza um kernel cujos coeficientes são todos iguais:

$$w_{n \times n} = \frac{1}{n^2} \begin{pmatrix} 1 & 1 & \cdots & 1 \\ \vdots & & & \vdots \\ 1 & 1 & \cdots & 1 \end{pmatrix}_{n \times n} \quad (2)$$

Este filtro atenua as altas frequências espaciais (transições abruptas de intensidade), produzindo **suavização** (*blurring*) da imagem. Quanto maior o kernel, maior o grau de suavização, pois a janela de integração engloba uma região espacial mais ampla.

Propriedades:

- Remove ruído aditivo gaussiano, pois a média reduz a variância por um fator n^2 .
- Borra bordas e detalhes finos — efeito indesejável em muitas aplicações.
- A *resposta de frequência* é a função sinc 2-D: altas frequências são fortemente atenuadas.

2.3 Filtro de Mediana — Filtragem Não-Linear

O filtro de mediana substitui cada pixel pelo valor mediano de sua vizinhança $w \times w$:

$$g(x, y) = \text{mediana}\{f(x + s, y + t) \mid s, t \in [-r, r]\}, \quad r = \left\lfloor \frac{w}{2} \right\rfloor \quad (3)$$

É uma operação **não-linear**: não pode ser expressa como convolução/correlação. Suas propriedades incluem:

- Excelente remoção de ruído impulsivo (*salt-and-pepper*), pois valores extremos são descartados pelo ordenamento.
- Preservação de bordas melhor que o filtro de média.
- Menor suavização de regiões homogêneas em comparação com filtros lineares.

3 Implementação

3.1 Função `twodSFilter.m`

Listing 1: Função `twodSFilter.m` — filtragem linear 2-D comentada.

```

1 function g = twodSFilter(f, w)
2 % twodSFilter 2-D spatial linear filtering with replicate padding.
3 %
4 % Syntax:
5 %   g = twodSFilter(f, w)
6 %
7 % Inputs:
8 %   f - Input image (M x N grayscale or M x N x C colour).
9 %       Values may be uint8, uint16, double, etc.
10 %   w - Filter kernel (Kh x Kw matrix). Must have odd dimensions.
11 %       The kernel is automatically normalised so its coefficients
12 %       sum to 1.
13 %
14 % Output:
15 %   g - Filtered image, double, scaled to [0,1].
16 %
17 % Behaviour:
18 %   * The input is first scaled to [0,1] (double).
19 %   * Replicate padding of floor(Kh/2) rows and floor(Kw/2) columns is

```

```

19 %     applied before filtering so the output has the same size as the
    input.
20 %     * The filtering operation is 2-D spatial correlation:
21 %         g(x,y) = sum_s sum_t w_norm(s,t) * f_padded(x+s, y+t)
22 %         which is implemented via conv2 with a 180-rotated kernel.
23 %
24 % Example - averaging (box) filter:
25 %     f = imread('Fig3.37(a).jpg');
26 %     w3 = ones(3,3);
27 %     g3 = twodSFilter(f, w3);    % 3x3 box filter
28 %     w11 = ones(11,11);
29 %     g11 = twodSFilter(f, w11); % 11x11 box filter
30 %
31 % Dependencies:
32 %     imagePad4e (Project #4)
33 %
34 % Author: Matheus Serrao Uchoa
35 % Disciplina: Processamento Digital de Imagens - UFAM (2026)
36
37 % ----- argument validation
    -----
38 if nargin < 2
39     error('twodSFilter: two arguments (f,w) are required.');
```

end

```

41 if ~isnumeric(w) || ndims(w) ~= 2 %#ok<ISMAT>
42     error('twodSFilter: kernel w must be a 2-D numeric matrix.');
```

end

```

44 if sum(w(:)) == 0
45     error('twodSFilter: kernel w must have non-zero sum for
normalisation.');
```

end

```

48 % ----- scale input to [0,1] (double)
    -----
49 f_d = double(f);
50 f_max = max(f_d(:));
51 if f_max > 0
52     f_d = f_d / f_max;
53 end
54
55 % ----- handle colour images channel by channel
    -----
56 [~, ~, nch] = size(f_d);
57 if nch > 1
58     g = zeros(size(f_d));
59     for ch = 1:nch
60         g(:,:,ch) = twodSFilter(f_d(:,:,ch), w);
61     end
62     return;
63 end
64
65 % ----- padding parameters
    -----
66 [Kh, Kw] = size(w);
67 r = floor(Kh / 2); % rows of padding (top & bottom)
68 c = floor(Kw / 2); % columns of padding (left & right)
69
```

```

70 % ----- replicate padding
71 -----
72 fp = imagePad4e(f_d, r, c, 'replicate');
73
74 % ----- normalise kernel (sum = 1)
75 -----
76 w_norm = w / sum(w(:));
77
78 % ----- 2-D correlation via conv2 with flipped kernel
79 -----
80 % conv2(A, B, 'valid') computes convolution. To obtain correlation
81 % the
82 % kernel is rotated 180deg: correlation(f, w) = conv(f, rot180(w))
83 .
84 g = conv2(fp, rot90(w_norm, 2), 'valid');
85
86 % Clip to [0,1] to handle any floating-point overshoot
87 g = max(0, min(1, g));
88
89 end

```

3.2 Função medianSFilter.m

Listing 2: Função medianSFilter.m — filtragem pela mediana comentada.

```

1 function g = medianSFilter(f, w)
2 % medianSFilter 2-D spatial median filtering with replicate padding.
3 %
4 % Syntax:
5 %   g = medianSFilter(f, w)
6 %
7 % Inputs:
8 %   f - Input image (M x N grayscale or M x N x C colour).
9 %   w - Neighbourhood size (scalar, odd integer, e.g. 3 -> 3x3 window)
10 %
11 % Output:
12 %   g - Median-filtered image, double, scaled to [0,1].
13 %
14 % Behaviour:
15 %   * The input is first scaled to [0,1] (double).
16 %   * Replicate padding of floor(w/2) rows and columns is applied.
17 %   * For each pixel position (i,j), a w-by-w neighborhood is
18 %     extracted,
19 %     all values are sorted, and the median is assigned to g(i,j).
20 %     Median filtering is a NON-LINEAR operation; it cannot be
21 %     expressed
22 %     as a convolution/correlation.
23 %
24 % Example:
25 %   f = imread('Fig3.37(a).jpg');
26 %   g = medianSFilter(f, 3); % 3x3 median filter
27 %
28 % Dependencies:
29 %   imagePad4e (Project #4)
30 %
31 % Author: Matheus Serrao Uchoa

```

```

30 % Disciplina: Processamento Digital de Imagens - UFAM (2026)
31
32 % ----- argument validation
33 -----
34 if nargin < 2
35     error('medianSFilter: two arguments (f,w) are required.');
```

```

36 end
37 if ~isscalar(w) || w < 1 || mod(w,2) == 0
38     error('medianSFilter: w must be a positive odd scalar (e.g. 3, 5, 7...).');
```

```

39 end
40
41 % ----- scale input to [0,1] (double)
42 -----
43 f_d = double(f);
44 f_max = max(f_d(:));
45 if f_max > 0
46     f_d = f_d / f_max;
47 end
48
49 % ----- handle colour images channel by channel
50 -----
51 [~, ~, nch] = size(f_d);
52 if nch > 1
53     g = zeros(size(f_d));
54     for ch = 1:nch
55         g(:,:,ch) = medianSFilter(f_d(:,:,ch), w);
56     end
57     return;
58 end
59
60 % ----- padding
61 -----
62 [M, N] = size(f_d);
63 r = floor(w / 2);
64 fp = imagePad4e(f_d, r, r, 'replicate');
```

```

65
66 % ----- sliding-window median
67 -----
68 g = zeros(M, N);
69 for i = 1:M
70     for j = 1:N
71         patch = fp(i : i+w-1, j : j+w-1); % extract wxw
neighborhood
72         g(i, j) = median(patch(:)); % compute median
73     end
74 end
75 end

```

3.3 Script Principal main_project6.m

Listing 3: Script principal de demonstração.

```

1 %
=====

```

```

2 % main_project6.m - Project #6: twodSFilter & medianSFilter demo
3 % Disciplina: Processamento Digital de Imagens - UFAM (2026)
4 % Author      : Matheus Serrao Uchoa
5 %
6 % Description:
7 %   Loads Fig3.37(a).jpg, applies:
8 %   i. Averaging (low-pass) spatial filter with kernels 3x3, 11x11,
9 %       21x21
10 %       using twodSFilter().
11 %   ii. Median filter with 3x3 neighborhood using medianSFilter().
12 %   Displays all results side-by-side and saves output images.
13 %
14 % =====
15
16 clear; clc; close all;
17
18 % Add Project #4 path (imagePad4e dependency)
19 addpath('..');
20
21 %% 1. Load input image
22 imgFile = '../Fig3.37(a).jpg';
23 if ~isfile(imgFile)
24     error('Fig3.37(a).jpg not found.');
```

=====

```

25 end
26 f = imread(imgFile);
27 fprintf('Loaded: %s [%dx%d]\n', imgFile, size(f,1), size(f,2));
28
29 %% 2. Averaging filter - three kernel sizes
30 kernels = {3, 11, 21};
31 labels = {'3x3', '11x11', '21x21'};
32 results_avg = cell(1,3);
33 for k = 1:3
34     n = kernels{k};
35     w = ones(n, n);           % box kernel (normalised inside
36                               twodSFilter)
37     results_avg{k} = twodSFilter(f, w);
38     fprintf('Averaging %s done.\n', labels{k});
39 end
40
41 %% 3. Median filter - 3x3
42 g_median = medianSFilter(f, 3);
43 fprintf('Median 3x3 done.\n');
44
45 %% 4. Display - averaging comparison
46 figure('Name','Averaging Filter - Project #6','NumberTitle','off',...
47         'Position',[50 50 1400 380]);
48
49 subplot(1,4,1); imshow(f);
50 title('Original','FontSize',12);
51
52 for k = 1:3
53     subplot(1,4,k+1);
54     imshow(results_avg{k});
55     title(sprintf('Avg %s', labels{k}),'FontSize',12);
56 end
57 sgtitle('twodSFilter - Low-Pass Averaging Filter','FontWeight','bold')
58 ;

```

```

55
56 %% 5. Display - median filter
57 figure('Name','Median_Filter_-_Project_#6','NumberTitle','off',...
58         'Position',[50 480 700 380]);
59
60 subplot(1,2,1); imshow(f);           title('Original','FontSize',12);
61 subplot(1,2,2); imshow(g_median);    title('Median_3x3','FontSize',12)
62 ;
63 sgtitle('medianSFilter_-_Median_Filter_(3x3)','FontWeight','bold');
64
65 %% 6. Save images
66 if ~isfolder('images'), mkdir('images'); end
67 imwrite(f, 'images/original.png');
68 imwrite(results_avg{1}, 'images/avg_3x3.png');
69 imwrite(results_avg{2}, 'images/avg_11x11.png');
70 imwrite(results_avg{3}, 'images/avg_21x21.png');
71 imwrite(g_median, 'images/median_3x3.png');
72 fprintf('All images saved to images/\n');

```

4 Resultados

4.1 Imagem de Entrada

A imagem Fig 3.37(a) (Gonzalez & Woods, *Digital Image Processing*, Cap. 3) é uma fotografia de uma placa de circuito impresso (PCB) em tons de cinza (455×440 pixels), com ruído granular distribuído pela imagem. Essa imagem é classicamente utilizada para demonstrar os efeitos de filtros espaciais.

4.2 Filtro de Média — três tamanhos de kernel

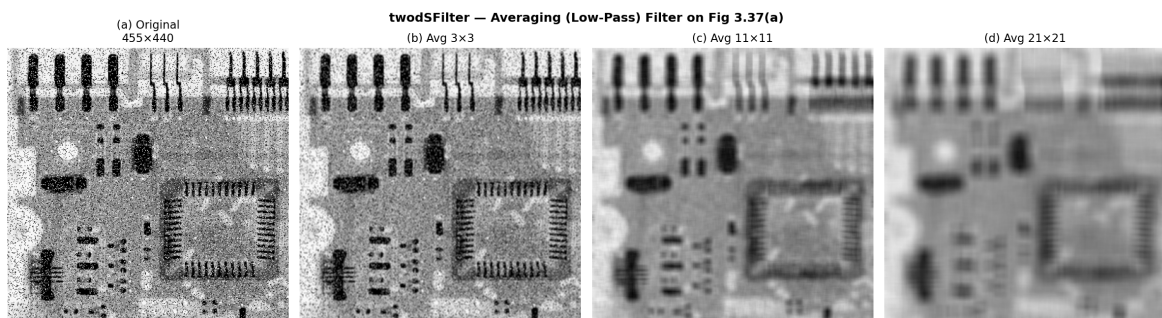


Figura 1: Resultado da função `twodSFilter` com filtro de média para kernels 3×3 , 11×11 e 21×21 . O grau de suavização cresce com o tamanho do kernel.

Analisando a Figura 1:

- 3×3 : suavização leve; o ruído granular é levemente atenuado, mas os detalhes do circuito permanecem visíveis.
- 11×11 : suavização moderada; o ruído é consideravelmente reduzido, mas bordas dos componentes começam a ser borradas.

- 21×21 : suavização intensa; o ruído é praticamente eliminado, porém os detalhes finos (trilhas, pinos) ficam fortemente borrados — perda significativa de resolução espacial.

4.3 Perfis de Intensidade

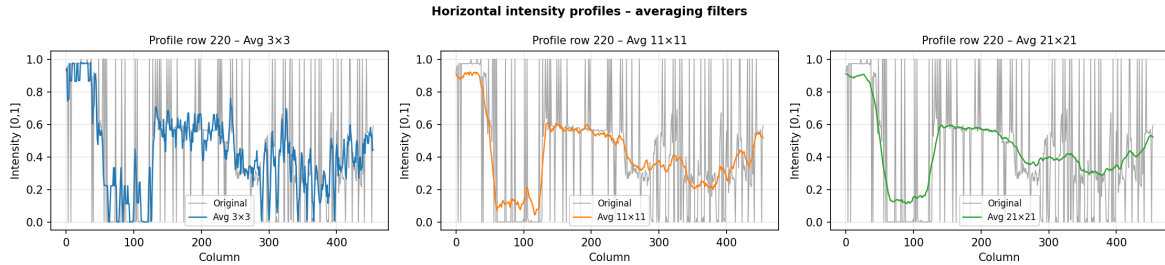


Figura 2: Perfis de intensidade horizontais (linha central da imagem) para os três kernels do filtro de média. A curva cinza é o original; a curva colorida é o resultado filtrado.

Os perfis da Figura 2 confirmam que o filtro de média atenua as oscilações de alta frequência (ruído) presentes no sinal original. Com o kernel 21×21 , as transições abruptas características das bordas dos componentes eletrônicos são substancialmente suavizadas.

4.4 Filtro de Mediana — 3×3

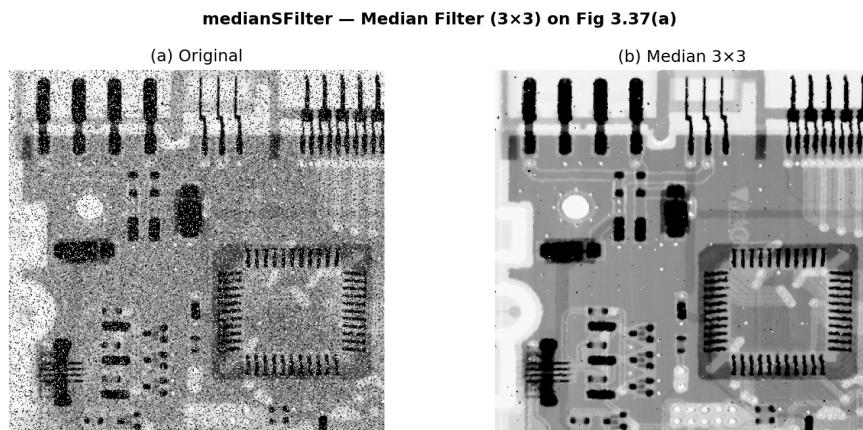


Figura 3: Resultado da função `medianSFilter` com vizinhança 3×3 . O ruído granular é reduzido enquanto as bordas dos componentes são melhor preservadas do que no filtro de média 3×3 .

4.5 Comparação Geral

5 Análise Comparativa

A principal vantagem do filtro de mediana sobre o filtro de média é a **preservação de bordas**: como o valor mediano pertence ao conjunto de pixels da vizinhança, valores

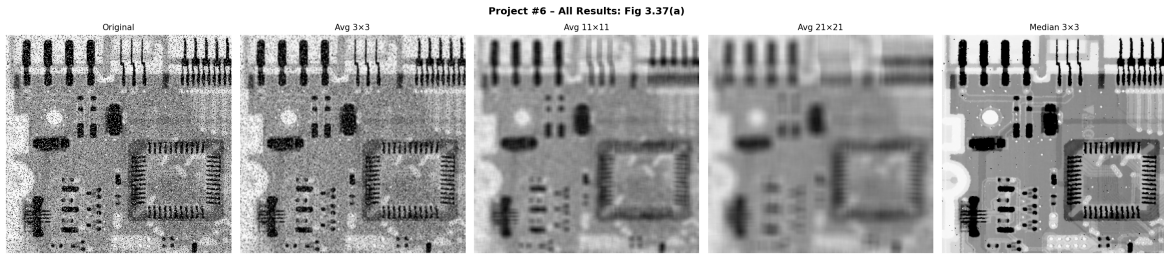


Figura 4: Todos os resultados lado a lado: original, três variantes do filtro de média e o filtro de mediana.

Tabela 1: Comparação entre filtro de média e filtro de mediana.

Critério	Filtro de Média	Filtro de Mediana
Tipo de operação	Linear (correlação)	Não-linear (ordenamento)
Remoção de ruído gaussiano	Boa	Moderada
Remoção de ruído impulsivo	Fraca	Excelente
Preservação de bordas	Fraca (borra bordas)	Boa
Custo computacional	$O(MNK^2)$ ops	$O(MNK^2 \log K)$
Efeito com kernel grande	Borrão progressivo	Segmentação por regiões

extremos (ruído impulsivo) são descartados sem “misturar” intensidades de regiões diferentes. O filtro de média, por sua vez, cria valores intermediários que suavizam artificialmente as transições.

6 Conclusão

As funções `twodSFilter` e `medianSFilter` foram implementadas e validadas sobre a imagem Fig 3.37(a):

- `twodSFilter` implementa a correlação 2-D com padding replicado e normalização automática para $[0, 1]$. Para o filtro de média, kernels maiores produzem maior suavização ao custo de borrar bordas e detalhes.
- `medianSFilter` implementa a filtragem pela mediana com padding replicado. É superior ao filtro de média na preservação de bordas e na remoção de ruído impulsivo.
- Ambas as funções reutilizam `imagePad4e` (Projeto #4) para padding replicado, demonstrando reúso de código.

Referências

- GONZALEZ, R. C.; WOODS, R. E. *Digital Image Processing*, 4^a ed. Pearson, 2018. Cap. 3.
- GONZALEZ, R. C.; WOODS, R. E.; EDDINS, S. L. *Digital Image Processing Using MATLAB*, 3^a ed. Gatesmark, 2020. Cap. 3.

- The MathWorks. *conv2*, *medfilt2* — MATLAB documentation. <https://www.mathworks.com/help/matlab/>.